

MULTIMODEL AI RESEARCH ASSISTANT

Capstone-II project report submitted

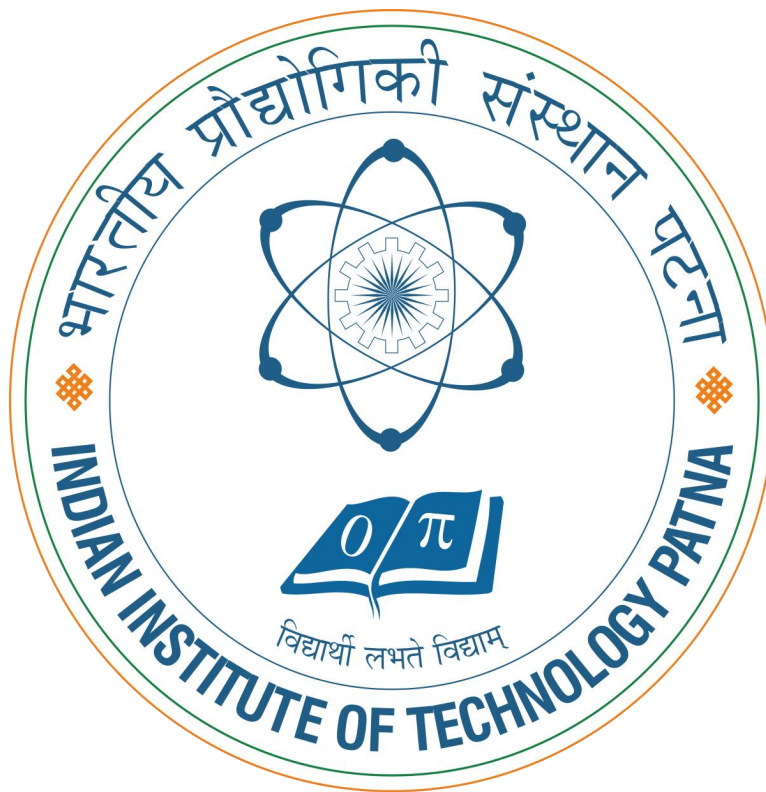
by the student of

UG program in Computer Science & Data Analytics

RUPESH KUMAR THAKUR

Roll No : 2312res844

Group No : 77



INDIAN INSTITUTE OF TECHNOLOGY PATNA

BIHTA - 801106, INDIA

30/03/2026

Declaration

I hereby declare that this submission is my own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Date: 30/03/2026



Rupesh Kumar Thakur

Roll No: 2312res844

Group No: 77

Summary of the Project:

“This project focuses on building an intelligent system that can read multiple PDF documents and answer user questions based on their content. The system uses Natural Language Processing (NLP) and Large Language Models (LLMs) to provide accurate and context-based answers read research papers(PDF) extract figures , tables math answer question using tech stack LLM(llama/OpenAI/Gemini), PDF parsing, summarization, dashboard UI.

The main goal of this project is to automate information retrieval from PDFs and improve the efficiency of searching and understanding large documents.

A full-stack research assistant with a ChatGPT-style frontend and a FastAPI backend for PDF-grounded question answering using Retrieval-Augmented Generation (RAG).

<https://multimodal-ai-research-assistant.vercel.app/>

Mention the work done by each member :

1. Rupesh Kumar Thakur(ME):

Led the frontend development and assisted in deployment and hosting-related tasks. I also participated in system testing and interface improvements during the mid-semester phase.

2. Swetankan Kumar Sinha :

led the backend development and system integration. His work includes the design and implementation of the RAG pipeline, FastAPI backend development, vector database integration using FAISS, and overall system coordination. He also contributed to frontend-backend integration and technical documentation.

3. Purushottam Kumar Singh :

Developed an initial prototype based on LangChain for PDF-based question answering, which helped in early-stage idea validation. He also contributed to the frontend development and assisted in refining the user interface of the current system.

4. Anurag Kumar Thakur :

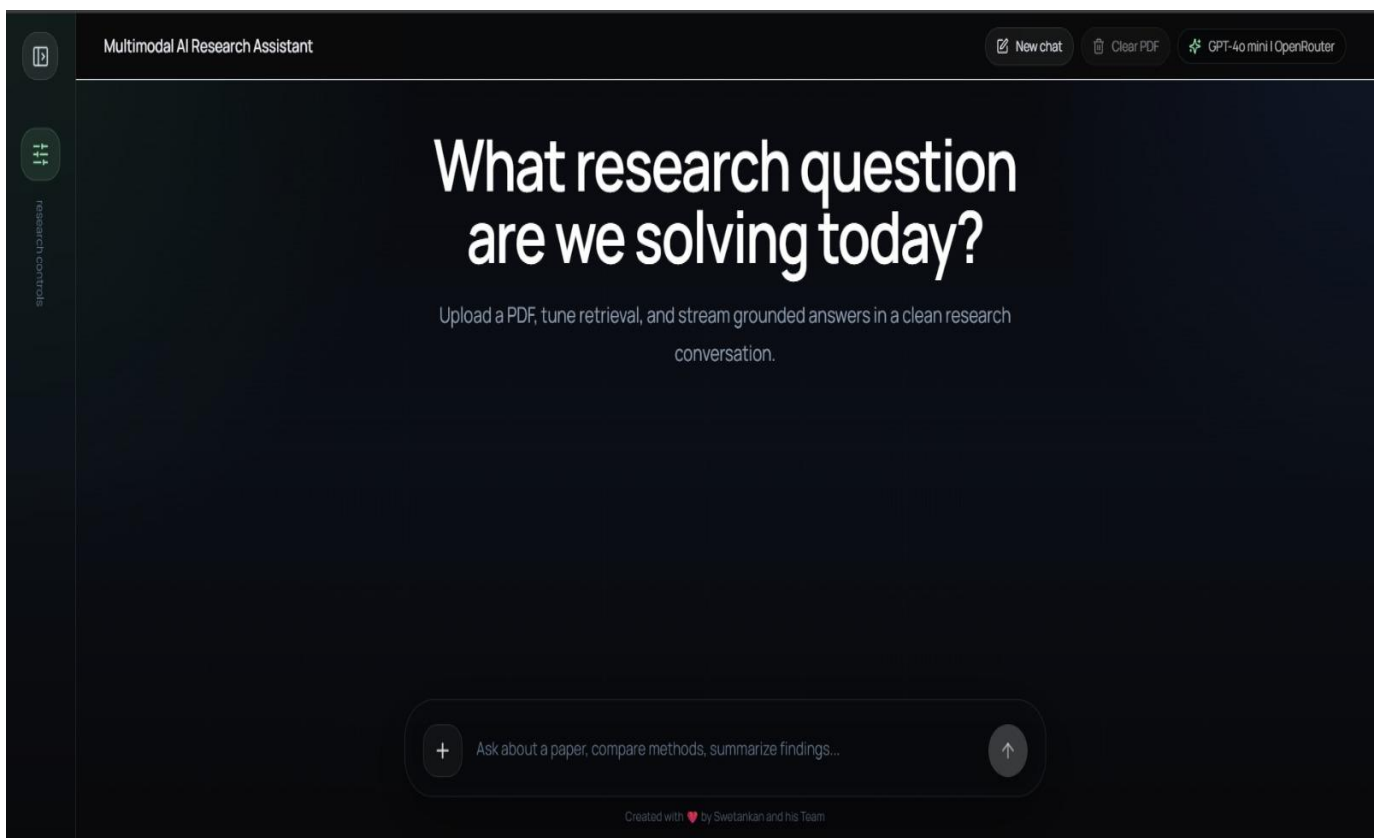
contributed to frontend support and testing of the system. He was involved in reviewing system behavior, identifying issues, and assisting in improving usability.

5. Shivam Kumar Singh :

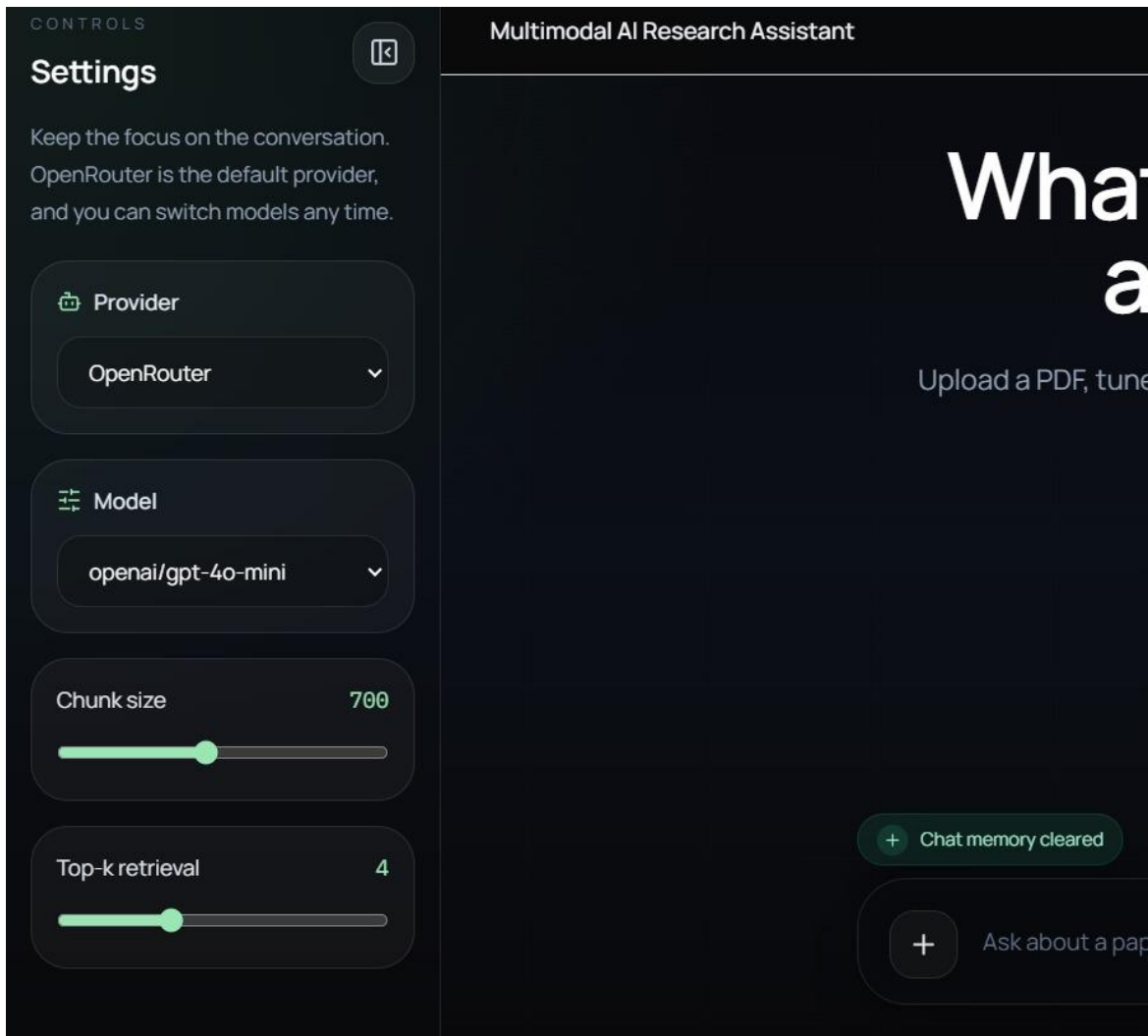
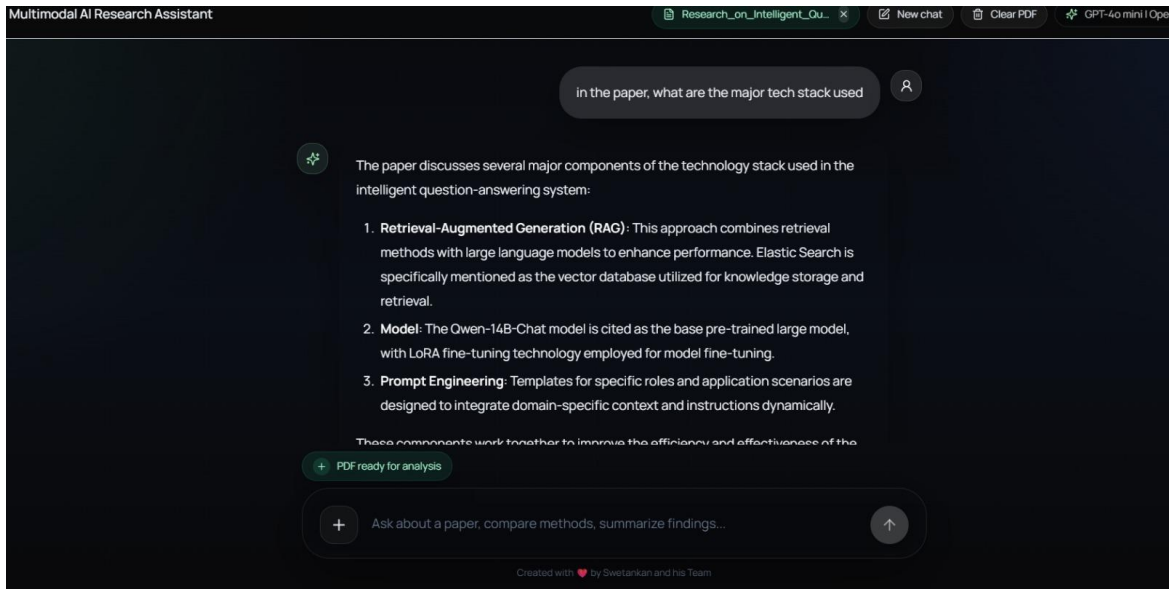
He has neither actively contributed to the current implementation phase of the project nor participated in any meeting held for discussion of the project, His contribution is expected in the remaining stages, particularly in documentation and evaluation

Contents

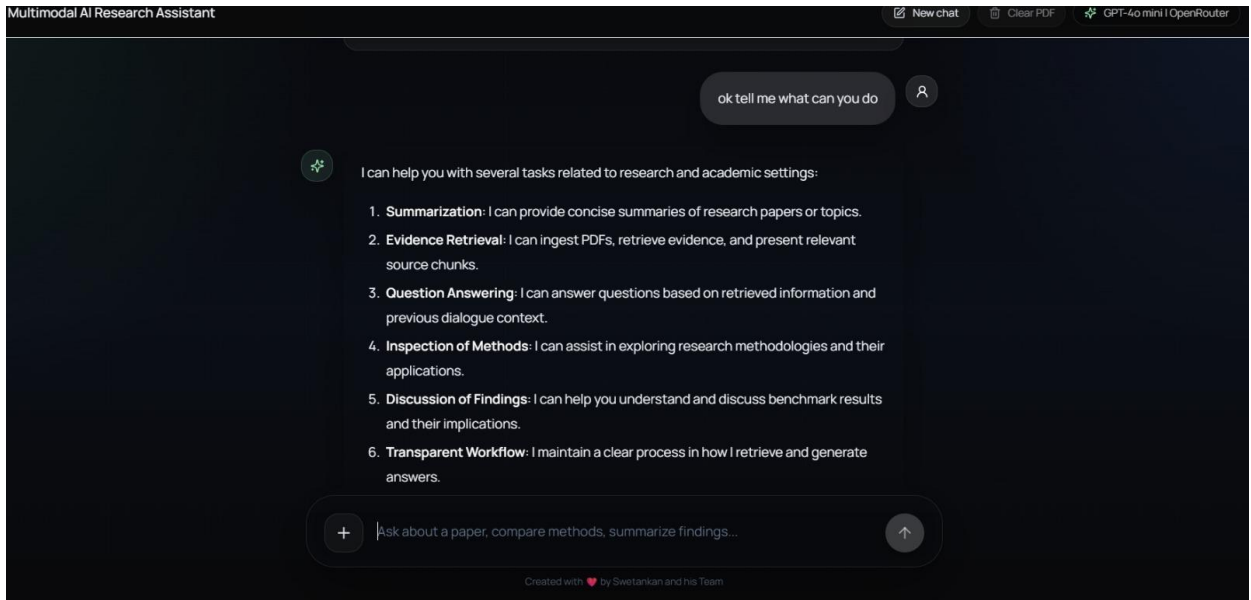
- 1 Introduction
- 2 Objective
- 3 Tech Stack
- 4 Features
- 5 Methodology
- 6 Future work
- 7 References



Contents



Contents



These techniques collectively aim to enhance the usability and user-friendliness of the system, making it more effective in delivering accurate and contextual responses.

Sources

RELEVANCE 0.645

t impact on system performance, demonstrating that external knowledge is crucial for mitigating model hallucination, and t impact on system performance, demonstrating that external knowledge is crucial for mitigating model hallucination, and its absence leads to a loss of professional knowledge provision capability within the system. Model fine -tuning can enhance complex reasoning abilities to solve the problem of "insufficient generalization" of domain knowledge in base LLM. Prompt engineering can improve the standardization and usability of answers to achieve the transition from "usable" to " user-friendly". V. CONCLUSION This paper constructs an intelligent question -answering system for the power industry that integrates LLMs and RAG

PDF ready for analysis

Introduction:

This project focuses on building an intelligent system that can read multiple PDF documents and answer user questions based on their content. The system uses Natural Language Processing (NLP) and Large Language Models (LLMs) to provide accurate and context-based answers.

The main goal of this project is to automate information retrieval from PDFs and improve the efficiency of searching and understanding large documents.

The system lets a user:

- upload a research paper in PDF format
- ask natural-language questions about that paper
- retrieve relevant chunks from the indexed document
- generate grounded answers through an LLM provider
- view the retrieved source chunks used to support the answer.

The Current implementation is optimized for a strong single -user capstone demonstration and viva discussion.

Objectives

- To upload and store multiple PDF files
- To extract and process text from PDFs
- To divide text into smaller meaningful chunks
- To store processed data in a vector database
- To retrieve relevant information based on user queries
- To generate accurate answers using LLM

2.Tech Stack

Frontend:

- Next.js App Router
- React
- Tailwind CSS
- custom component-based chat UI
- markdown and code rendering support

Backend:

- FastAPI
- FAISS

- sentence-transformers
- httpx
- pypdf
- python-dotenv

LLM providers:

- OpenRouter by default
- Ollama optional

Observability and quality:

- LangSmith tracing
- pytest test suite
- retrieval accuracy check script

Features

- ChatGPT-like dark conversation UI
- dynamic greeting on first load
- sticky bottom input bar
- inline PDF upload from the composer
- streaming assistant responses
- thinking indicator and live streaming state
- markdown rendering
- styled code blocks with copy support
- collapsible source attribution section
- sidebar controls for provider, model,

- chunk size, and top-k
- identity handling for developer attribution
- reset controls for new chat and PDF clearing
- retrieval debug and retrieval evaluation endpoints
- backend test suite

3. Methodology

3.1 PDF Upload and Storage

- The system allows uploading multiple PDF files.
- PDFs are stored in a designated folder.
- It supports handling multiple documents at the same time.

3.2 Text Extraction

- Text is extracted from uploaded PDF files.
- Raw text is collected for further processing.

3.3 Text Cleaning

- Remove unnecessary symbols, noise, and extra spaces.

- Improve readability and structure of text.
 - Prepare clean data for better processing.
-

3.4 Text Chunking

- Large text is divided into smaller chunks.
 - Chunk size and overlap are carefully selected.
 - Overlapping helps maintain context between chunks.
 - Optimized chunking improves answer quality.
-

3.5 Vector Database Storage

- Each text chunk is converted into vector embeddings.
 - These embeddings are stored in a vector database.
 - It enables fast and accurate similarity search.
-

3.6 Question Answering Process

Step 1: User Input

- The system takes a question from the

user.

Step 2: Retrieval

- Relevant chunks are retrieved from the vector database.

Step 3: Prompt Design

- A structured prompt is created using the retrieved data.

Step 4: Answer Generation

- The LLM generates an answer based on the prompt.

Step 5: Optimization

- Improve answer quality, relevance, and clarity.

4. Key Features

- Handles multiple PDFs
- Efficient text extraction and cleaning
- Smart chunking with context preservation
- Fast retrieval using vector database
- Accurate answers using LLM
- Improved response quality

Conclusion & future work

6. Conclusion

This project successfully demonstrates how PDF documents can be transformed into an intelligent question-answering system. By combining text processing, vector databases, and LLMs, the system provides accurate and context-aware answers.

It reduces manual effort and improves efficiency in handling large amounts of textual data.

7. Future Work:

Multi-document support: The assistant can be extended to index multiple papers simultaneously and support metadata-aware retrieval across a paper collection.

Persistent vector database: A more robust vector database such as Chroma Qdrant, Weaviate, or PostgreSQL with vector

extensions could replace the current local persistence model

Hybrid search (BI25 + embeddings): A stronger hybrid stack can improve retrieval precision on lexical and method-specific queries.

Reranking models: Adding a cross-encoder or reranker can improve the quality of the final retrieved chunk set, especially for nuanced questions such as limitations or ablations.

Research paper comparison module: The assistant can be extended to compare methods, datasets, metrics, and contributions across multiple uploaded papers.

Figure and table understanding: Future versions can parse tables and images more explicitly, allowing better handling of

chat-based or benchmark-heavy documents.

PPT generation: A useful extension would be automatic generation of presentation slides or viva summaries from a paper and chat history.

Migration to LangGraph for advanced workflows: If the system evolves into a multi-step research agent with tools, web search, planning, and memory, LangGraph would become a more justified orchestration layer.

Answer evaluation and benchmark datasets: Future work should include a stronger evaluation suite with ground-truth answer sets, retrieval precision at k, citation faithfulness, and user studies.

Session and user isolation: Multi-user architecture with document namespaces.

References

- <https://vercel.com/docs/frameworks/full-stack/nextjs>
- <https://vercel.com/docs/git/vercel-for-github>
- <https://render.com/docs/deploy-fastapi>

Fundamental Of Large language Model(LLM)by
Tong Xiao& Jingbo Zho. NLP by Sriparna Saha
mam class content.

THANKU